

# **Maybe Attention Is Not All You Need**

Linear-Time Language Modeling via  
Wave Propagation and Resonance Memory

Sage Architecture  
2026

## Abstract

The Transformer architecture, underpinned by the self-attention mechanism, has dominated natural language processing for the better part of a decade. However, its  $O(N^2)$  computational and memory complexity with respect to sequence length severely limits its applicability to very long contexts and resource-constrained hardware. In this paper, we introduce Sage, an entirely attention-free, linear-time  $O(N)$  language modeling architecture. Sage replaces softmax-based attention with a dual-mechanism approach: Wave Propagation (multi-scale causal convolutions) for resolving local syntax, and Resonance Memory (a causally accumulated neural whiteboard) for global semantic understanding. We demonstrate that this architecture achieves competitive perplexity while operating at strict  $O(N)$  complexity, enabling full model training and fast inference on consumer-grade hardware with as little as 4GB VRAM.

# 1. Introduction

Since the landmark paper "Attention Is All You Need" (Vaswani et al., 2017), the self-attention mechanism has been the de facto standard for sequence modeling in NLP, computer vision, and beyond. By computing pairwise dot-products between all tokens in a sequence, attention provides an unrestricted global receptive field in a single layer. However, this powerful mechanism comes at a steep cost: it requires computing and storing an  $N \times N$  attention matrix, imposing a severe quadratic scaling bottleneck on both compute and memory.

Recent efforts to mitigate this include sparse attention patterns (Beltagy et al., 2020), linear attention approximations (Katharopoulos et al., 2020), and state-space models such as Mamba (Gu & Dao, 2023). Building upon these principles, we present an alternative formulation that decomposes context processing into two explicitly separate mechanisms: one for local syntactic structure and one for global semantic retrieval.

We propose the Sage Architecture, which replaces the  $O(N^2)$  self-attention block entirely with a combination of multi-scale causal convolutions (Wave Propagation) and a low-rank cumulative memory matrix (Resonance Memory). The result is a model that is strictly  $O(N)$  in sequence length while maintaining strong language modeling capabilities.

## 2. Architecture Overview

The Sage architecture abandons the Query-Key-Value (QKV) paradigm. Instead, information flows through a stack of repeating WaveBlocks. Each WaveBlock consists of three sequential sub-modules, connected via pre-norm residual connections:

- Wave Propagation (WaveMixer) — local syntactic processing
- Resonance Memory — global semantic retrieval
- SwiGLU MLP — position-wise reasoning and transformation

Let  $X \in \mathbb{R}^{(B \times L \times D)}$  be the input tensor, where  $B$  is batch size,  $L$  is sequence length, and  $D$  is the model dimension. The forward pass through a single WaveBlock is defined as:

$$\begin{aligned} X_1 &= X + \text{WaveMixer}(\text{RMSNorm}(X)) \\ X_2 &= X_1 + \text{ResonanceMemory}(\text{RMSNorm}(X_1)) \\ X_{\text{out}} &= X_2 + \text{SwiGLU}(\text{RMSNorm}(X_2)) \end{aligned}$$

**The full model wraps  $N$  such blocks between an embedding layer and an LM head:**

$$\text{tokens} \rightarrow \text{Embedding}(\text{vocab}, D) \rightarrow [\text{WaveBlock} \times N] \rightarrow \text{RMSNorm} \rightarrow \text{Linear}(D, \text{vocab}) \rightarrow \text{logits}$$

Weight tying is applied between the embedding matrix and the LM head projection, reducing total parameter count by nearly 50% in small-scale configurations.

## 3. Wave Propagation (Local Context Mechanism)

### 3.1 Motivation

Standard attention dedicates immense parameter capacity to modeling local word dependencies (e.g., verb-noun agreement, determiner-noun pairings) which are inherently proximity-based. These local patterns can be captured far more efficiently by causal convolutions, which process tokens within a fixed-size sliding window at  $O(N \cdot k)$  cost, where  $k$  is the kernel size.

### 3.2 Multi-Scale Design

The WaveMixer splits the embedding dimension  $D$  into three non-overlapping chunks and processes each with a differently-sized 1D depthwise causal convolution:

- Short-range ( $d_{\text{short}} = D // 4$ ): kernel size  $k_1$  — captures immediate adjacency (bigrams, trigrams)
- Mid-range ( $d_{\text{mid}} = D // 4$ ): kernel size  $k_2$  — captures phrasal structure
- Long-range ( $d_{\text{long}} = D - d_{\text{short}} - d_{\text{mid}}$ ): kernel size  $k_3$  — captures clause-level dependencies

**Kernel sizes are adaptive per layer depth:**

```
depth_ratio = layer_idx / max(n_layers - 1, 1)
k_short = 3
k_mid = 5 + int(depth_ratio * 12)    # range: 5 → 17
k_long = 11 + int(depth_ratio * 32)   # range: 11 → 43
# Both k_mid and k_long are forced to be odd (bitwise OR with 1)
```

### 3.3 Causal Padding

To maintain strict auto-regressive causality, each convolution is left-padded by  $(k - 1)$  zeros along the sequence dimension. This ensures that the output at position  $t$  depends only on inputs at positions 0 through  $t$ . No positional embeddings are required since position information is implicitly encoded in the causal structure.

### 3.4 Gated Output

After concatenating the three convolution outputs back to dimension  $D$ , the result is passed through a sigmoid-gated linear projection:

```
Y_concat = Concat([Y_short, Y_mid, Y_long], dim=-1)
output =  $\sigma(\text{gate\_proj}(Y_{\text{concat}})) \odot \text{value\_proj}(Y_{\text{concat}})$ 
```

where `gate_proj` and `value_proj` are both `Linear(D, D, bias=False)`.

### 3.5 Complexity Analysis

Each depthwise convolution operates in  $O(N \cdot k)$  time and  $O(D \cdot k)$  parameters. Since  $k$  is a fixed constant ( $\leq 43$  in the deepest layer), the total complexity is strictly  $O(N)$ , independent of sequence length. This contrasts sharply with attention's  $O(N^2)$  scaling.

## 4. Resonance Memory (Global Context Mechanism)

### 4.1 Motivation

While Wave Propagation handles local context efficiently, language understanding also requires global information routing — e.g., resolving coreferences across distant paragraphs or maintaining topic coherence over thousands of tokens. In standard Transformers, this is handled by the  $N \times N$  attention matrix. Resonance Memory provides the same capability at  $O(N)$  cost.

### 4.2 Mechanism Overview

The Resonance Memory operates in four stages:

- Write: Each token projects its state into  $K$  shared memory slots with learned importance weights.
- Accumulate: Memory is built causally via  $O(N)$  cumulative sum over the sequence dimension.
- Read: Each token generates a read query to retrieve relevant context from accumulated state.
- Gate: A learned sigmoid gate controls how much retrieved context mixes with the current state.

### 4.3 Detailed Tensor Operations

Hyperparameters:  $K = 16$  (number of slots),  $\text{mem\_dim} = 32$  (compressed memory dimension).

#### Step 1 — Write Keys and Values:

```
W_key = Softmax(Linear(D → K)(X), dim=-1)    # shape: (B, L, K)
W_val = Linear(D → mem_dim)(X)                 # shape: (B, L, 32)
```

#### Step 2 — Memory Update Tensor:

```
U = W_key.unsqueeze(-1) × W_val.unsqueeze(-2) # shape: (B, L, K, 32)
```

#### Step 3 — Causal Accumulation:

```
M = cumsum(U, dim=1)                          # shape: (B, L, K, 32)
```

This single cumsum operation is the key insight: position  $t$ 's memory state  $M[t]$  contains the sum of all writes from positions 0 through  $t$ , enabling global information access in  $O(N)$  time.

#### Step 4 — Read and Retrieve:

```
R_key = Softmax(Linear(D → K)(X), dim=-1)    # shape: (B, L, K)
R_compressed = Σ_K(R_key.unsqueeze(-1) × M)   # shape: (B, L, 32)
R_full = RMSNorm(Linear(32 → D)(R_compressed)) # shape: (B, L, D)
```

#### Step 5 — Gated Fusion:

```
gate = σ(Linear(2D → D)(Concat[X, R_full]))
output = X + gate ⊙ R_full
```

### 4.4 Complexity Analysis

The total cost is  $O(N \cdot K \cdot \text{mem\_dim})$  per layer. With  $K=16$  and  $\text{mem\_dim}=32$ , this amounts to only 512 multiply-accumulate operations per token per layer — negligible compared to attention's  $O(N \cdot D)$  per

token. The 4D memory tensor occupies approximately 8 MB per layer at batch size 8, sequence length 128, compared to hundreds of megabytes for a full attention KV-cache at equivalent scale.

#### 4.5 Comparison with Attention

| Property         | Self-Attention             | Resonance Memory                                  |
|------------------|----------------------------|---------------------------------------------------|
| Time Complexity  | $O(N^2 \cdot D)$           | $O(N \cdot K \cdot d_{\text{mem}})$               |
| Space Complexity | $O(N^2)$ attention matrix  | $O(N \cdot K \cdot d_{\text{mem}})$ cumsum tensor |
| Global Access    | Full pairwise (all-to-all) | Via accumulated memory slots                      |
| Causality        | Requires explicit mask     | Inherent via cumulative sum                       |

### 5. SwiGLU Feed-Forward Network

The position-wise transformation uses the SwiGLU activation (Shazeer, 2020), following the implementation used in LLaMA-3 and other modern architectures:

```
W1 = Linear(D → mlp_dim, bias=False)
W2 = Linear(mlp_dim → D, bias=False)
W3 = Linear(D → mlp_dim, bias=False)
```

```
output = W2(SiLU(W1(x))) ⊙ W3(x)
```

The default `mlp_dim` is set to  $2 \times \lceil (8D / 3) / 2 \rceil$ , following the 8/3 ratio convention. For  $D=512$ , this yields `mlp_dim=1365`.

### 6. Normalization: RMSNorm

All normalization layers use Root Mean Square Layer Normalization (Zhang & Sennrich, 2019) instead of standard LayerNorm. RMSNorm omits the mean-centering step, retaining only the variance-based rescaling:

$$\text{RMSNorm}(x) = x / \sqrt{(\text{mean}(x^2) + \epsilon)} \times \gamma$$

where  $\gamma$  is a learnable scale parameter and  $\epsilon = 1e-6$ .

### 7. Metacognitive Adaptive Depth

Sage includes an optional iterative refinement mechanism. After the initial forward pass through all WaveBlocks, the model can re-process the representation through the block stack additional times (up to `max_think_iterations`). An early stopping criterion based on representation convergence (cosine similarity between successive iterations) determines when further processing yields diminishing returns.

```
for i in range(max_iterations):
    x_new = forward_through_blocks(x)
    if cosine_similarity(x, x_new) > threshold:
```

```

        break
    x = x_new

```

## 8. Full Model Assembly (PyTorch Reference)

The following is a minimal reference implementation of the complete model:

```

class SageLM(nn.Module):
    def __init__(self, vocab_size, dim, n_layers):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, dim)
        self.blocks = nn.ModuleList([
            WaveBlock(dim, layer_idx=i)
            for i in range(n_layers)
        ])
        self.norm = RMSNorm(dim)
        self.lm_head = nn.Linear(dim, vocab_size, bias=False)
        self.lm_head.weight = self.embed.weight # weight tying

    def forward(self, token_ids):
        x = self.embed(token_ids) # (B, L) → (B, L, D)
        for block in self.blocks:
            x = block(x) # (B, L, D) → (B, L, D)
        return self.lm_head(self.norm(x)) # (B, L, D) → (B, L, V)

```

## 9. Default Configuration

| Parameter                          | Value                     |
|------------------------------------|---------------------------|
| <b>Model Dimension (D)</b>         | 512                       |
| <b>MLP Dimension</b>               | 1365                      |
| <b>Number of Layers</b>            | 6                         |
| <b>Vocabulary Size</b>             | 100,277 (cl100k_base BPE) |
| <b>Resonance Slots (K)</b>         | 16                        |
| <b>Memory Dimension</b>            | 32                        |
| <b>Max Think Iterations</b>        | 3                         |
| <b>Think Convergence Threshold</b> | 0.92                      |
| <b>Total Parameters</b>            | ~31.2 Million             |

|                 |        |
|-----------------|--------|
| Checkpoint Size | 119 MB |
|-----------------|--------|

## 10. Training Details and Experimental Results

### 10.1 Training Configuration

- Optimizer: AdamW ( $\beta_1=0.9$ ,  $\beta_2=0.999$ , weight\_decay=0.01)
- Learning Rate:  $3e-4$  with cosine annealing and linear warmup (500 steps)
- Precision: Automatic Mixed Precision (Float16 via torch.amp)
- Batch Size: 8
- Sequence Length: 128 tokens
- Gradient Clipping: max\_norm = 1.0
- Dataset: 15,000 instruction-following conversations (ShareGPT)
- Tokenizer: cl100k\_base BPE (tiktoken)
- Hardware: NVIDIA RTX 3050 (4GB VRAM)

### 10.2 Results

| Epoch | Train Loss | Val Loss | Val Perplexity |
|-------|------------|----------|----------------|
| 1     | 5.95       | 5.69     | 296            |
| 5     | 4.55       | 5.18     | 178            |
| 10    | 4.05       | 5.00     | 148            |
| 15    | 2.85       | 4.97     | 145            |

**Inference Speed: 21–23 tokens/sec on RTX 3050 (4GB VRAM).**

**Peak VRAM Usage: < 4 GB during both training and inference.**

## 11. Conclusion

This work challenges the prevailing assumption that  $O(N^2)$  context mixing is necessary for language understanding. By decoupling local syntax processing (Wave Propagation) from global context retrieval (Resonance Memory), the Sage architecture provides an  $O(N)$  linear-time alternative to the Transformer. Our preliminary validation demonstrates that sophisticated causal reasoning architectures can be trained and deployed entirely within ultra-low resource environments, opening avenues for on-device language models, edge computing, and long-context applications that are currently infeasible with standard Transformer architectures.